



UNIVERSIDAD DON BOSCO

FACULTAD DE INGENIERIA

ESCUELA DE COMPUTACION

Guía de laboratorio Nº 6

Ciclo I

Nombre de la práctica: Manejos de hojas de estilo con DOM.

Lugar de ejecución: Centro de cómputo.

Materia: Desarrollo de Aplicaciones Web con Software Interpretado en el Cliente.

I Objetivos.

- ✓ Implementar la utilización de Hojas de Estilo en Cascada.
- ✓ Identificar las propiedades y métodos principales para el manejo de Hojas de Estilo desde un script.
- ✓ Hacer uso de las propiedades y métodos de estilo del DOM para modificar dinámicamente un documento web.
- ✓ Realizar aplicaciones con HTML Dinámico (DHTML) modificando propiedades de estilo de forma dinámica utilizando JavaScript.

II. Introducción Teórica.

Hojas de Estilo en Cascada

Las Hojas de Estilo en Cascada son un estándar para especificar la presentación de documentos HTML o XML. De acuerdo a este estándar, los elementos HTML deberían utilizarse solamente para definir la estructura del documento, dejando la tarea de la presentación y formato del mismo a las Hojas de Estilo. Esto significa que deberían evitarse a toda costa, la utilización de elementos HTML desaprobados como FONT, CENTER, B, I, U y los atributos HTML relacionados con la presentación, como bgcolor, align, bordercolor, etc.

CSS (*Cascading Style Sheets*) es una tecnología diseñada para ser utilizada por los diseñadores web o cualquiera preocupado por la precisión visual de los documentos HTML. Es vital su utilización desde JavaScript debido a que el Modelo de Objetos de Documento (DOM) permite gestionar, dinámicamente, a través de secuencias de comando, los estilos aplicados a los elementos individuales de un documento. La utilización conjunta de CSS y JavaScript permite crear una variedad de efectos visuales conocidos como HTML Dinámico (DHTML).

Formas de aplicación de estilos en un documento

Antes que nada, hay que recordar cómo se pueden aplicar estilos en un documento. Básicamente son tres formas:

1. Estilos en línea

Se aplica una o varias reglas de estilo de forma directa en la etiqueta, haciendo uso del atributo style, válido para diversos elementos HTML.

Ejemplo:

```
<p style="color:green;border:solid red 2px;">Este párrafo tiene el texto verde y un borde rojo.</p>
```

2. Estilos internos

Es una hoja de estilo que se incrusta dentro del documento HTML dentro del elemento STYLE que debe estar en la cabecera del documento (HEAD). Debe proporcionar atributos adicionales al elemento, entre los que se destacan: rel, type, href y media.

Ejemplo:

```
<style type="text/css">
h1 {
  border-width:1;
  border:solid;
  text-align:center }
</style>
```

3. Estilos externos

Es una hoja de estilo que está almacenada en un archivo .css independiente y que se vincula al documento donde se quiere aplicar, haciendo uso del elemento LINK, especificando en su atributo href el documento .css que contiene la definición de estilos para el documento.

Ejemplo:

```
<link rel="stylesheet" type="text/css" href="misestilos.css" />
```

Donde se ha utilizado sintaxis XHTML.

El archivo misestilos.css podría contener un código CSS como el siguiente:

```
h1 {
  font-family:"Bookman Old Style";
  font-weight:bold;
  font-size:20pt;
  font-style:Italia;
  color:Green;
  text-align:center;
}
p {
  font:italic lighter 1em Garamond;
  width:80%;
  margin:auto;
  text-align:justify;
}
.frase {
  font:normal bold 0.8em Georgia;
  text-align:center;
}
```

CSS y DHTML

Los desarrolladores web de contenido DHTML, deben poner principal atención en la posibilidad de utilizar atributos CSS normales para especificar la visibilidad, el tamaño y la posición exacta de elementos individuales dentro de un documento. Otras propiedades CSS permiten definir el orden de apilamiento, la transparencia, el área de recorte, los márgenes, el relleno, los bordes y los colores. Para programar con DHTML, es importante saber cómo funcionan estas propiedades. La siguiente tabla resume la mayor parte de estas:

Atributo	Descripción
position	Especifica el tipo de posicionamiento aplicado a un elemento.
top	Indica la posición del borde superior de un elemento. El valor de la medida puede ser indicado en forma relativa o absoluta.
left	Indica la posición del borde izquierdo de un elemento. El valor de la medida puede ser indicado en forma relativa o absoluta.
bottom	Indica la posición del borde inferior de un elemento. El valor de la medida puede ser indicado en forma relativa o absoluta.
right	Indica la posición del borde derecho de un elemento. El valor de la medida puede ser indicado en forma relativa o absoluta.
width, height	Especifican el tamaño de un elemento. El ancho y el alto respectivamente.
z-index	Indica el orden de apilamiento de un elemento con relación a cualquier otro elemento superpuesto, permitiendo establecer una tercera dimensión para indicar la posición del elemento.
display	Especifica cómo se va a presentar un elemento en el documento y, más aún, si se presentará o no.
visibility	Indica si un elemento será visible o no en el documento. El elemento seguirá ocupando un espacio físico en el documento, independientemente de si es visible o no.

clip	Define un área de recorte para un elemento; sólo se muestran dentro de esta área partes del elemento. Especifica lo que se debe hacer si un elemento es mayor que el espacio asignado para éste.
margin	Especifica el margen que el área de contenido del elemento tendrá con respecto a los border del mismo.
border	Indica el ancho del borde de un elemento y la apariencia de este.
background	Especifica el color o la imagen de fondo a utilizar en un elemento, permitiendo definir otras características adicionales de este fondo, como si se repetirá la imagen hasta llenar el fondo, si se mantendrá o no fija cuando se desplace el contenido de la ventana, etc.
opacity	Especifica la opacidad (o transparencia) de un elemento. Este es un atributo de CSS3 admitido en varios navegadores actuales.

La clave del DHTML: el posicionamiento absoluto

El atributo *position* del CSS, indica el tipo de posicionamiento aplicado a un elemento. Los valores posibles para este atributo son cuatro:

- ✓ **static:** es el valor predeterminado y se utiliza para indicar que el elemento debe colocarse de acuerdo al flujo normal del contenido del documento (en casi todas las aplicaciones esto significa que debe colocarse de izquierda a derecha o de arriba hacia abajo, mientras exista espacio para el mismo). Los elementos ubicados de forma estática no pueden ser elementos DHTML, lo que significa que no se pueden definir atributos como *top*, *left*, *right* o *bottom* para posicionarlos de forma dinámica.
- ✓ **absolute:** permite establecer la posición del elemento con respecto a su elemento contenedor. Los elementos ubicados de forma absoluta se colocan independientemente del espacio utilizado por el resto de elementos presentes en el documento y no formará parte del flujo normal. Un elemento posicionado de forma absoluta se ubica en el documento con relación a su elemento contenedor. Con frecuencia este elemento contenedor es el elemento BODY, pero si el elemento está anidado dentro de otro elemento, entonces, este elemento será con respecto al que se ubicará. Este es el tipo de posicionamiento utilizado con mayor frecuencia para aplicaciones DHTML, de hecho es el más recomendado por razones de compatibilidad entre navegadores. También, es preciso asegurarse de colocar los elementos posicionados de forma absoluta dentro de elementos DIV o SPAN para asegurarse de su funcionamiento apropiado en navegadores Internet Explorer.
- ✓ **fixed:** permite definir la posición de un elemento con respecto a la ventana del explorador web. En teoría, los elementos con posición fija siempre serán visibles y no se desplazarán con el resto de elementos presentes en el documento cuando se utilicen barras de desplazamiento o teclado para acceder a otras partes de la página web. Al igual que los elementos posicionados de forma absoluta, los elementos con posición fija se ubican de forma independiente del resto de elementos y no forman parte del flujo normal del documento. El Internet Explorer 6.0 e inferiores no soportan el posicionamiento fijo.
- ✓ **relative:** este tipo de posicionamiento hace que un elemento se coloque, primero, según el flujo normal del documento. Luego, se ajusta su ubicación con relación a su posición en el flujo normal. El espacio asignado para este elemento en el flujo normal, sigue siendo ocupado, aunque el elemento ya no esté en esa posición. Estos significan que el resto de elementos en el flujo normal no pueden ocupar esta posición. Una vez que se posiciona el elemento con uno de los tres valores que hagan posible realizar aplicaciones de HTML dinámico (*absolute*, *fixed* o *relative*), puede establecerse su posición con una combinación de los atributos ***left***, ***top***, ***right*** y ***bottom***. La técnica de posicionamiento más común es utilizar los atributos ***left*** y ***top***. Por ejemplo, para posicionar un elemento a 60 píxeles de la parte izquierda y a 75 píxeles de la parte superior del documento, debe especificarse una regla de estilo como la siguiente:

```
selector {
    position: absolute;
    left: 60px;
    top: 75px;
}
```

El DOM y las CSS

Las Hojas de Estilo es otro de los aspectos soportados por la mayor parte de los navegadores actuales. Es el DOM Nivel 2 el que permite manipular los valores de las propiedades CSS de los elementos presentes en un documento.

Manipulación de estilos insertados en línea

La forma más sencilla de manipular valores de CSS con JavaScript es a través de la propiedad style que corresponde a la especificación de hojas de estilo insertado en línea para un elemento HTML específico.

Por ejemplo:

```
<p id="myParagraph">Esto es una prueba</p>
```

Insertando el estilo en línea:

```
<p id="myParagraph" style="color:brown">Esto es una prueba</p>
```

Para manipular las interfaces del DOM con JavaScript puede utilizar un script como este:

```
theElement = document.getElementById("myParagraph");
```

```
theElement.style.color = "green";
```

En el ejemplo anterior puede verse que se ha utilizado la propiedad style del elemento cuyo id fue definido como "myParagraph". Luego, se estableció la propiedad color del objeto style como "green". Esta propiedad color se corresponde exactamente con la propiedad color de las CSS. El punto clave es, por tanto, saber a qué propiedad del DOM hacer referencia para modificar una propiedad de estilo en particular. La tabla siguiente muestra esta correspondencia:

Propiedad CSS	Propiedad DOM Nivel 2
background	background
background-attachment	backgroundAttachment
background-color	backgroundColor
background-image	backgroundImage
background-position	backgroundPosition
background-repeat	backgroundRepeat
border	border
border-color	borderColor
border-style	borderStyle
border-top	borderTop
border-right	borderRight
border-left	borderLeft
border-bottom	borderBottom
border-top-color	borderTopColor
border-right-color	borderRightColor
border-bottom-color	borderBottomColor
border-left-color	borderLeftColor
border-top-style	borderTopStyle
border-right-style	borderRightStyle
border-bottom-style	borderBottomStyle
border-left-style	borderLeftStyle
border-top-width	borderTopWidth
border-right-width	borderRightWidth
border-bottom-width	borderBottomWidth
border-left-width	borderLeftWidth
border-width	borderWidth
clear	clear
clip	clip
color	color

display	display
float	cssFloat
font	font
font-family	fontFamily
font-size	fontSize
font-style	fontStyle
font-variant	fontVariant
font-weight	fontWeight
height	height
left	left
letter-spacing	letterSpacing
line-height	lineHeight
list-style	listStyle
list-style-image	listStyleImage
list-style-position	listStylePosition
list-style-type	listStyleType
margin	margin
margin-top	marginTop
margin-right	marginRight
margin-bottom	marginBottom
margin-left	marginLeft
overflow	overflow
padding	padding
padding-top	paddingTop
padding-right	paddingRight
padding-bottom	paddingBottom
padding-left	paddingLeft
position	position
text-align	textAlign
text-decoration	textDecoration
text-indent	textIndent
text-transform	textTransform
top	top
vertical-align	verticalAlign
visibility	Visibility
white-space	whiteSpace
width	width
word-spacing	WordSpacing
z-index	zIndex

Estilo dinámico utilizando clases y colecciones

Mediante la utilización de selectores de clase es posible aplicar un conjunto de reglas de estilo a elementos iguales o diferentes. En el siguiente ejemplo se muestran dos clases definidas en un elemento STYLE:

```
<style type="text/css">
.On {
    background-color:LightSkyBlue;
    color:Crimson;
    border:5px Crimson groove;
}

.Off {
    background-color:LightSteelBlue;
    color:DarkBlue;
```

```

        border:5px DarkBlue ridge;
    }
</style>

```

La idea es definir un elemento HTML como perteneciente a una de las clases. Tomemos la clase `.Off` y hagamos que el elemento `h1` sea de esa clase. Haciéndolo desde HTML estático, tendríamos que crear una etiqueta agregando un atributo `class`, como se muestra a continuación:

```
<h1 id="encendido" class="Off">Poner el t&iacute;tulo</h1>
```

Para manipular el aspecto del elemento de encabezado `h1`, puede hacer uso de JavaScript, redefiniendo la clase (**class**) del elemento. El valor del atributo **class** puede manipularse haciendo uso de la propiedad del DOM denominada **className**, de la siguiente forma:

```
theElement = document.getElementById("encendido");
theElement.className = "On";
```

En la práctica estos nombres de clase se redefinen haciendo uso del manejo de eventos de manera que al pasar el ratón por encima (**onmouseover**) del elemento se le da un nombre y al retirarlo (**onmouseout**) se le devuelve el nombre original, con el que se nombró al elemento al cargar la página.

```
<h1 id="encendido" class="Off" onmouseover="this.className='On'"
    onmouseout="this.className='Off'">Poner el t&iacute;tulo</h1>
```

Otra forma de manipular estilos es utilizar el método `getElementByTagName()` y efectuar los cambios de estilo sobre cada uno de los elementos individuales devueltos. En el procedimiento se mostrará un ejemplo de cómo emplear este método.

Acceso a reglas de estilo complejas

El DOM Nivel 2, permite tener acceso a la colección `styleSheets[]` que forma parte del objeto `Document`. A través de esta colección se puede tener acceso a las diversas etiquetas `<style>` y `<link>` de un documento. Para acceder a los estilos definidos en estas etiquetas se puede hacer uso de una instrucción como cualquiera de las siguientes:

```
var firstStyleSheet = document.styleSheets[0];
```

o también:

```
var firstStyleSheet = document.styleSheets.item(0);
```

La manipulación de las propiedades del objeto correspondiente al elemento `STYLE` hallado en el documento HTML es directa, ya que sus propiedades se corresponden con la de los atributos HTML que ya conocemos. Como: `type`, `href`, `media`, `disabled`, entre otros.

En el DOM, el objeto **CSSStyleSheet** hereda las características del objeto **StyleSheet** y luego añade la colección **cssRules[]**, que contiene las diversas reglas del bloque de estilos, así como los métodos **insertRule()** y **deleteRule()**. Cabe destacar que el Internet Explorer sigue su propio modelo para manipular estilos. Para este navegador habrá que hacer uso de métodos similares, que son: **addRule()** y **removeRule()** para el objeto **styleSheet**.

La sintaxis del método **insertRule()** es **theStyleSheet.insertRule("texto_regla_estilo", indice)**. Donde, **"texto_regla_estilo"** es una cadena que contiene el selector de hojas de estilo y la declaración. En tanto que, **indice** es la posición para insertarlo dentro del conjunto de reglas. La sintaxis del método **addRule()**, es similar y tiene la siguiente estructura:

```
theStyleSheet.addRule("h3","color:blue;font-size:12pt;",4);
```

El acceso a las reglas individuales es inmediato al hacer uso de la colección **cssRules[]**, o para Internet Explorer, con la colección no estándar **rules[]**. Una vez que se accede a una regla, se puede establecer o modificar su propiedad `selectorText` para examinar el selector de una regla en particular. También se puede utilizar la propiedad `style` para acceder al conjunto real de reglas.

El DOM nivel 2 proporciona diversos métodos para modificar reglas de estilo. Por ejemplo: `getPropertyValue()` y `setProperty()`. Sin embargo, es aconsejable acceder a estas a través del objeto `style` y luego, a la propiedad DOM que corresponda a la propiedad CSS en cuestión. Por ejemplo:

```
theStyleSheet.cssRules[0].style.color = 'blue'; //modifica o añade una propiedad color a
//la primera regla CSS de la hoja de estilos
theStyleSheet.rules[0].style.color = 'blue'; //Utilice esta en Internet Explorer
```

III Materiales y Equipos.

No.	Requerimiento	Cantidad
1	Guía de práctica #5: Manejo de Hojas de Estilo con el DOM	1
2	Computadora con editor HTML y navegadores instalados	1

IV. Procedimiento.

Ejercicio #1: Probador de estilos mediante un campo de selección de formulario, controlando con JavaScript no invasivo los estilos que se aplican a un elemento h1. La secuencia de comando es capaz de agregar más de una clase, permitiendo además de agregar clases, eliminarlas para manejar de forma apropiada el hecho de que un elemento puede tener más de una clase. Si la clase ya fue asignada, la clase no se vuelve a agregar.

Guión 1: testclasses.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Cambio de clases CSS dinámico</title>
  <meta charset="utf-8" />
  <script src="js/classname.js"></script>
  <link rel="stylesheet" href="css/headings.css">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTWfSpd3yD65VohhpuuCOMLASjC" crossorigin="anonymous">
</head>
<body class="container">
<header>
  <h1 id="head">Texto de prueba estilos con className</h1>
</header>
<section>
<article>
<div class="mb-3" >
  <label>Estilo:</label>
  <select id="classes" name="classes" class="form-select">
    <option value="" selected="selected">(Escoja un estilo)</option>
    <option value="formal">Formal</option>
    <option value="letterpress">Texto incrustado</option>
    <option value="offset">Sombra desplazada</option>
  </select><br />

  <div class="btn-group" role="group" aria-label="Basic mixed styles example">
    <button type="button" class="btn btn-danger" name="agregar" id="add"
value="Agregar">Agregar</button>
    <button type="button" class="btn btn-warning" name="eliminar" id="remove"
value="Eliminar">Eliminar</button>
  </div>
</div>
</article>
</section>
</body>
</html>
```

Guión 2: headings.css

```
.formal {
  background:#57758a;
  color:#fff;
  font-family:'LeagueGothic-Regular', sans-serif;
  font-size:3.6em;
  padding: 6px 9px;
  text-align: center;
  text-shadow: 0px -1px 0px #303030;
}
```

```

.letterpress{
  color: #222;
  background:#414141;
  font-family:'Chunkfive', sans-serif;
  font-size:3.2em;
  padding-right: 12px;
  text-align: right;
  text-shadow: 0px 2px 1px #555;
}

.offset{
  background:#e4e4c7;
  /* Dejar suficiente espacio de relleno para acomodar la sombra */
  padding:16px 16px 25px 16px;
  font-family:'LeagueGothic-Regular', sans-serif;
  font-size:3.8em;
  letter-spacing:5px;
  font-weight:bold;
  color:#526972;
  text-align: left;
  text-shadow:4px 4px 0px #e4e4c7,
              6px 6px 0px #526972;
}

```

Guión 3: classname.js

```

function init(){
  var btnagregar = document.getElementById("add");
  var btneliminar = document.getElementById("remove");
  var hElement = document.getElementById("head");
  var clsName;
  alert(hElement + " | " + clsName);
  //Controlar evento click sobre el botón Agregar
  if(btnagregar.addEventListener){
    btnagregar.addEventListener("click", function(){
      clsName =document.getElementById("classes").value;
      if(clsName=="formal"){
        hElement.classList.add('formal');
      }else if(clsName=="letterpress"){
        hElement.classList.add('letterpress');
      }else if("offset"){
        hElement.classList.add('offset');
      }
    }, false);
  }
  else if(btnagregar.attachEvent){
    btnagregar.attachEvent("onclick", function(){
      clsName =document.getElementById("classes").value;
      if(clsName=="formal"){
        hElement.classList.add('formal');
      }else if(clsName=="letterpress"){
        hElement.classList.add('letterpress');
      }else if("offset"){
        hElement.classList.add('offset');
      }
    });
  }
  //Controlar evento click sobre botón Eliminar
  if(btneliminar.addEventListener){
    btneliminar.addEventListener("click", function(){
      clsName =document.getElementById("classes").value;
      if(clsName=="formal"){
        hElement.classList.remove('formal');
      }else if(clsName=="letterpress"){
        hElement.classList.remove('letterpress');
      }else if("offset"){
        hElement.classList.remove('offset');
      }
    }, false);
  }
}

```

```

    }
    else if (btneliminar.attachEvent) {
        btneliminar.attachEvent("onclick", function () {
            clsName = document.getElementById("classes").value;
            if (clsName == "formal") {
                hElement.classList.remove('formal');
            } else if (clsName == "letterpress") {
                hElement.classList.remove('letterpress');
            } else if ("offset") {
                hElement.classList.remove('offset');
            }
        });
    }
}

if (window.addEventListener) {
    window.addEventListener("load", init, false);
}
else if (window.attachEvent) {
    window.attachEvent("onload", init);
}

```

Resultado:

Texto de prueba estilos con
className

Estilo:

Sombra desplazada

Agregar Eliminar

Ejercicio #2: Agregar, modificar, habilitar y deshabilitar reglas de estilo usando el DOM para manejo dinámico de Hojas de Estilo.

Guión 1: estilosDOM.html

```

<!DOCTYPE html>
<html lang="es">
<head>
    <title>Manejo de reglas de estilo con el DOM</title>
    <meta charset="utf-8" />
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztqcQTWfspd3yD65VohhpuuCOMLASjC" crossorigin="anonymous">
</head>
<body class="container">
<header>
    <h1>Documento de prueba CSS</h1>
    <p>Esto es un párrafo de <strong>prueba</strong>.</p>
    <p>Aquí viene más texto <em>engañoso</em>.</p>
    <p>Hecho. No necesitamos <strong>seguir</strong> con esto</p>
</header>
<section>
<article>
<h3>Fin del documento de prueba</h3>
<div class="btn-group" role="group" aria-label="Basic mixed styles example">
    <input type="button" class="btn btn-danger" value="Habilitar" id="enabled">
    <input type="button" class="btn btn-warning" value="Deshabilitar" id="disabled">
    <input type="button" class="btn btn-success" value="Modificar regla" id="change">
    <input type="button" class="btn btn-info" value="Borrar regla" id="remove">
    <input type="button" class="btn btn-secondary" value="Añadir regla" id="add">
</div>
</article>
</section>
<script src="js/estilosdom.js"></script>

```

```
</body>
</html>
```

Guión 2: estilosdom.js

```
function init(){
    var btnenabled = document.getElementById("enabled");
    var btndisabled = document.getElementById("disabled");
    var btnchange = document.getElementById("change");
    var btnremove = document.getElementById("remove");
    var btnadd = document.getElementById("add");
    //Implementación del botón Habilitar reglas de estilo
    if(btnenabled.addEventListener){
        btnenabled.addEventListener("click", function(){
            document.styleSheets[0].disabled = false;
        }, false);
    }
    else if(btnenabled.attachEvent){
        btnenabled.attachEvent("onclick", function(){
            document.styleSheets[0].disabled = false;
        });
    }
    //Implementación del botón deshabilitar reglas de estilo
    if(btndisabled.addEventListener){
        btndisabled.addEventListener("click", function(){
            document.styleSheets[0].disabled = true;
        }, false);
    }
    else if(btndisabled.attachEvent){
        btndisabled.attachEvent("onclick", function(){
            document.styleSheets[0].disabled = true;
        });
    }
    //Implementación del botón modificar regla
    if(btnchange.addEventListener){
        btnchange.addEventListener("click", modifyRule, false);
    }
    else if(btnchange.attachEvent){
        btnchange.attachEvent("onclick", modifyRule);
    }
    //Implementación del botón borrar o quitar regla
    if(btnremove.addEventListener){
        btnremove.addEventListener("click", deleteRule, false);
    }
    else if(btnremove.attachEvent){
        btnremove.attachEvent("onclick", deleteRule);
    }
    //Implementación del botón añadir regla
    if(btnadd.addEventListener){
        btnadd.addEventListener("click", addRule, false);
    }
    else if(btnadd.attachEvent){
        btnadd.attachEvent("onclick", addRule);
    }
}

function modifyRule(){
    var styleSheet = document.styleSheets[0];
    if(styleSheet.rules){
        styleSheet.cssRules = styleSheet.rules;
    }
    if(styleSheet.cssRules[0]){
        styleSheet.cssRules[0].style.color = 'purple';
        styleSheet.cssRules[0].style.fontSize = '30pt';
        styleSheet.cssRules[0].style.backgroundColor = 'gold';
    }
}

function deleteRule(){
    var styleSheet = document.styleSheets[0];
```

```
//Haciendo compatible la propiedad rules
if(styleSheet.rules){
    styleSheet.cssRules = styleSheet.rules;
}
if(styleSheet.cssRules.length > 0){
    if(styleSheet.removeRule)
        styleSheet.removeRule(0);
    else if(styleSheet.deleteRule)
        styleSheet.deleteRule(0);
}
}

function addRule(){
    var styleSheet = document.styleSheets[0];
    if("insertRule" in styleSheet){
        styleSheet.insertRule('h3 {text-align:center; font-family:"Century Gothic"; font-size:18pt; color:Brown;}', 4);
    }
    else if("addRule" in styleSheet){
        styleSheet.addRule('h3', 'text-align:center; font-family:"Century Gothic"; font-size:18pt; color:Brown;', 4);
    }
}

if(window.addEventListener){
    window.addEventListener("load", init, false);
}
else if(window.attachEvent){
    window.attachEvent("onload", init);
}
```

Resultado:

Documento de prueba CSS

Esto es un párrafo de **prueba**.

Aquí viene más texto *engañoso*.

Hecho. No necesitamos **seguir** con esto

Fin del documento de prueba

Habilitar

Deshabilitar

Modificar regla

Borrar regla

Añadir regla

Ejercicio #3: Ventanas movibles. Con este ejemplo se ilustra una aplicación que permite mover sobre la pantalla dos pequeñas ventanas, validando límites de ventana dentro del navegador.

Archivo 1: `ventanas.css.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
    <title>Ventanas creadas con CSS</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="css/windows.css">
    <script src="js/loadevents.js"></script>
    <script src="js/drag.js"></script>
</head>
<body>
<!-- Crea una ventana de prueba con las CSS definida -->
<div class="window" style="left:10px;top:10px;z-index:10;">
<div class="titlebar" id="titlew1">
    Ventana de prueba
</div>
<div class="content">
    <p>
```

La paleta de colores RGB (RVA en español) consta, básicamente, de tres colores primarios aditivos: Rojo-Verde-Azul. Estos colores primarios aditivos, en HTML, están representados por tres pares hexadecimales del tipo 0xHH-HH-HH según el siguiente formato:

```
</p>
<p class="centrado">
  #RRGGBB (= #RRVVAA)
</p>
<p>
```

Los valores que puede adoptar cada uno de los tres pares hexadecimales van del 0x00 (0 decimal) al 0xFF (255 decimal). Cuanto mayor sea el valor del par, tanto mayor será también la intensidad (matiz, brillo o claridad) del color correspondiente a ese par (y viceversa). Esto implica que el extremo inferior de la escala cromática parte de una intensidad (grado) de color mínima (nulo = par 0x00), pasa por una intensidad de color media (mediano = par 0x80 [128 decimal]) hasta llegar a una intensidad de color máxima (saturado = par 0xFF). El grado de más alta pureza (absoluto) de un color primario aditivo estará determinado por la presencia total del mismo (saturación = 0xFF) junto con la ausencia total (nulidad = 0x00) de los otros dos colores primarios aditivos.

```
</p>
</div>
</div>
<!-- Crea otra ventana en otra posición superpuesta -->
<div class="window" style="left:75px;top:110px;z-index:20;">
<div class="titlebaractive" id="titlew2">
  Otra ventana
</div>
<div class="content translucent" style="background-color:#d0d0d0;font-weight:bold;">
  <p>
    Esta es otra ventana. Su valor <tt>z-index</tt> la pone encima
    de la otra ventana. Los estilos CSS hacen que su &acute;rea
    de contenido se vea transparente en los navegadores que soportan
    esta caracter&iacute;stica.
  </p>
  
</div>
</div>
</body>
</html>
```

Guión 2: drag.js

```
/* *****
 * drag.js: este script sirve para arrastrar elementos HTML *
 *           posicionados de forma absoluta.                *
 *                                                         *
 * El script define una sola función drag() diseñada para   *
 * llamarse desde un controlador de evento onmousedown. Los *
 * eventos mousemove siguientes se moverán al elemento     *
 * especificado. Un evento mouseup terminará el arrastre.   *
 * *****/
// elementToDrag es el elemento que recibe al evento mousedown
// o algún elemento contenedor. Tiene que colocarse de forma
// absoluta. Sus valores style.left y style.top se cambiarán
// basándose en el arrastre del usuario.
// event es el objeto event para el evento mousedown.
function drag(elementToDrag, event){
  //posición del ratón (en coordenadas de ventana) en la que
  //se inicia el arrastre.
  var startX = event.clientX, startY = event.clientY;
  //posición original (en coordenadas del documento) del
  //elemento que se va a arrastrar. Como elementToDrag se ha
  //posicionado de forma absoluta, suponemos que su offsetParent
  //es el cuerpo del documento.
  var origX = elementToDrag.offsetLeft, origY = elementToDrag.offsetTop;
  //Aunque las coordenadas se calculen en sistemas de coordenadas
  //diferentes, podemos calcular todavía la diferencia existente
```

```

//entre ellas y utilizarlas en la función moveHandler(). Esta
//acción funciona porque la posición de la barra de desplazamiento
//nunca cambia durante el arrastre.
var deltaX = startX - origX, deltaY = startY - origY;
//Registrar los controladores de eventos que van a responder a
//los eventos mousemove y al evento mouseup que sigue a este
//evento mousedown.
if(document.addEventListener){ //Modelo de eventos DOM Nivel 2
    document.addEventListener("mousemove", moveHandler, true);
    document.addEventListener("mouseup", upHandler, true);
}
else if(document.attachEvent){ //Modelo de eventos de IE 5+
    //En el modelo de eventos de IE, se capturan los eventos
    //llamando al método setCapture() en el elemento.
    elementToDrag.setCapture();
    elementToDrag.attachEvent("onmousemove", moveHandler);
    elementToDrag.attachEvent("onmouseup", upHandler);
    //Tratar la pérdida de captura del ratón como un evento mouseup
    elementToDrag.attachEvent("onlosecapture", upHandler);
}
else{
    //En IE 4 no se puede utilizar attachEvent() o setCapture(),
    //por lo que se establecen controladores de eventos directamente
    //en el objeto document y se espera a que los eventos del ratón
    //que se necesitan se propaguen.
    var oldmovehandler = document.onmousemove;
    var olduphandler = document.onmouseup;
    document.onmousemove = moveHandler;
    document.onmouseup = upHandler;
}
if(event.stopPropagation) event.stopPropagation(); //Nivel 2 del DOM
else event.returnValue = false; //IE

//Controlador que captura los eventos mousemove cuando se está
//arrastrando un elemento. Es el responsable de mover el elemento
function moveHandler(e){
    if(!e) e = window.event; //Modelo de eventos IE
    elementToDrag.style.left = (e.clientX - deltaX) + "px";
    elementToDrag.style.top = (e.clientY - deltaY) + "px";
    if(e.stopPropagation) e.stopPropagation(); //Nivel 2 del DOM
    else e.cancelBubble = true; //IE
}

//Controlador que captura el evento mouseup final que se produce
//al final del arrastre
function upHandler(e){
    var titleBar = document.getElementById('titlew1');
    var titleBarSecond = document.getElementById('titlew2');
    titleBar.className == 'titlebaractive' ? titleBar.className = 'titlebar' :
titleBar.className = 'titlebaractive';
    titleBarSecond.className == 'titlebar' ? titleBarSecond.className = 'titlebaractive'
: titleBarSecond.className = 'titlebar';
    if(!e) e = window.event; //Modelo de eventos de IE
    //Anular el registro de los controladores de eventos de captura
    if(document.removeEventListener){
        document.removeEventListener("mouseup", upHandler, true);
        document.removeEventListener("mousemove", moveHandler, true);
    }
    else if(document.detachEvent){ //Modelo de eventos IE
        elementToDrag.detachEvent("onlosecapture", upHandler);
        elementToDrag.detachEvent("onmouseup", upHandler);
        elementToDrag.detachEvent("onmousemove", moveHandler);
        elementToDrag.releaseCapture();
    }
    else{ //Modelo de eventos de IE 4
        //Restablecer los controladores originales
        document.onmouseup = olduphandler;
        document.onmousemove = oldmovehandler;
    }
    //No permitir que se propague más el evento
    if(e.stopPropagation) e.stopPropagation(); //Nivel 2 del DOM

```

```

    else e.cancelBubble = true;           //IE
  }
}

```

Guión 3: loadevents.js

```

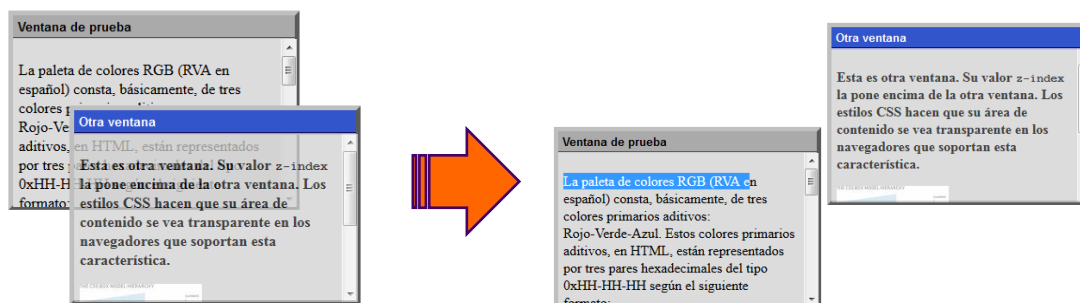
function init(){
  var divw1 = document.getElementById("titlew1");
  var divw2 = document.getElementById("titlew2");
  if(divw1.addEventListener){
    divw1.addEventListener("mousedown", function(event){
      drag(this.parentNode, event);
    }, false);
  }
  else if(divw1.attachEvent){
    divw1.attachEvent("onmousedown", function(event){
      drag(this.parentNode, event);
    });
  }
  if(divw2.addEventListener){
    divw2.addEventListener("mousedown", function(event){
      drag(this.parentNode, event);
    });
  }
  else if(divw2.attachEvent){
    divw2.attachEvent("onmousedown", function(event){
      drag(this.parentNode, event);
    });
  }
}

if(window.addEventListener){
  window.addEventListener("load", init, false);
}
else if(window.attachEvent){
  window.attachEvent("onload", init);
}

```

Agregar a su proyecto el archivo: windows.css proporcionado en los recursos de la guía.

Resultado: Después de cargar la página y mover las ventanas con el ratón por el área de la ventana del navegador.



V. Discusión de Resultados.

1. Realice una aplicación que grafique valores que miden el uso de un determinado navegador o sistema operativo. La aplicación muestra varios controles de tipo botón de opción de formulario para escoger los resultados que se desean graficar, el color de los gráficos y la orientación de las barras.

Gráfico de barras

Seleccione una gráfica:

- ☒ Uso de navegadores
- ☐ Uso de Sistemas Operativos

Elija un color:

- ☒ Red ☐ Verde ☐ Azul

Elija una dirección:

- ☐ Horizontal ☒ Vertical



Gráfico de barras

Seleccione una gráfica:

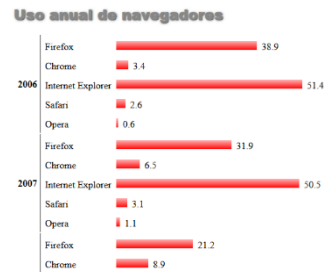
- ☒ Uso de navegadores
- ☐ Uso de Sistemas Operativos

Elija un color:

- ☒ Red ☐ Verde ☐ Azul

Elija una dirección:

- ☒ Horizontal ☐ Vertical



VI. Bibliografía.

- Deitel, Paul / Deitel, Harvey / Deitel, Abbey. Internet & World Wide Web. Cómo programar. 5a. Edición. Editorial Pearson. 2014. México D.F..
- John Resig / Bear Bibeault. JavaScript Ninja. 1a. Edición. Editorial Anaya Multimedia / Manning. Septiembre 2013. Madrid, España.
- Flanagan, David. JavaScript La Guía Definitiva. 1a Edición. Editorial ANAYA Multimedia. 2007. Madrid, España.
- Terry McNavage. JavaScript Edición 2012. 1a. Edición. Editorial ANAYA Multimedia. 2011. Madrid, España.
- Tom Negrino / Dori Smith. JavaScript & AJAX para diseño web. 6ta. Edición. Pearson/Prentice Hall. 2007. Madrid, España.